

SRE基础-2

云原生监控项目

2025.12-2026.01

云原生可观测性平台与自动化运维系统

负责人

- **初步实施**: `awk`、`top`、`free`、`df`采集 CPU、内存、磁盘利用率指标，阈值时触发脚本发送告警邮件，`crontab`定时采集；
- **系统升级**: 考虑到现代运维技术日渐成熟，将手动的监控脚本，全面升级为基于**Prometheus + Grafana + Alertmanager**的标准监控系统，实现指标采集、**可视化**到告警的自动化与集中化管理，提升运维效率；
- **云原生化与可靠性检验**: 通过 **Docker** 容器化封装所有组件，并使用 Docker Compose 进行编排，实现开发、测试、生产环境的一致性与一键启停，解决环境差异导致的部署问题。同时引入**混沌工程**实践，主动**模拟**CPU、内存等故障，验证监控告警链路的及时性与有效性，变被动响应为主动防御；
- **自动化部署**: 编写 **Ansible Playbook**，成功在多台服务器上的一键部署监控栈，部署效率提高90%。

面试官：你最初用 `crontab + awk + top` 做监控，这种方式有什么致命缺陷？

单机碎片化——每个服务器独立采集和判断，无法汇聚全局视图

无历史数据——只能看到当前瞬时值，无法分析趋势或追溯故障

告警能力原始——阈值判断简单粗暴，缺乏去抖动、聚合、静默等机制，容易产生告警风暴

维护成本高——脚本分散、版本混乱、调试困难，整体处于被动响应状态，与现代化监控的主动、自动、可视化理念相去甚远

你做混沌工程时，是怎么模拟 CPU 和内存故障的？注入故障后，你在 Linux 底层看到了什么现象？告警是怎么触发的？

我主要使用 `stress` 工具 和 `ChaosBlade`（阿里巴巴开源的混沌实验平台）来模拟故障，对于 CPU 故障，通过 `stress --cpu 4 --timeout 60s` 命令启动多个进程满负荷运行，使 CPU 使用率迅速飙升；对于内存故障，则使用 `stress --vm 2 --vm-bytes 2G --timeout 60s` 分配大量内存，快速消耗可用内存

现象：利用率100%，`load average`上升，`free -m`发现可用内存迅速下降

当指标超过预设阈值（如 CPU > 80% 持续 1 分钟）时，**Prometheus** 将告警推送到 **Alertmanager**，后者经过分组、抑制和路由，最终通过邮件或企业微信发送告警通知

Linux内核

模块一：负载模型

进程状态机

R状态-正在干活 或 随时准备干活，如果用 `top` 看到有大量的进程处于 R 状态，并且这个数量远远超过了你服务器的 CPU 核心数，说明CPU 资源严重不足，系统很可能会卡顿

S状态-可中断睡眠，在等通知，可随时叫醒，是绝大多数服务的常态

D状态-不可中断，进程正在等待底层的I/O操作，比如读写本地磁盘，等待NFS网络文件系统返回的情况，如果系统 Load 飙升到 100，但 CPU 使用率只有 5%，这绝对是因为系统里出现了大量 D 状态进程。虽然它们不消耗 CPU，但 Linux 的 Load Average 统计算法是包含 D 状态进程的。这通常意味着底层存储发生了严重故障（如磁盘坏道、I/O 性能打满）

Z状态-僵尸进程，进程已经执行完毕退出了，它释放了所有的内存和资源，为了让它的父进程知道退出状态码是多少，内核会保留它的一个PID，正常情况下，父进程会立刻调用 `wait()` 或 `waitpid()` 把这个状态码取走，子进程就彻底消失了。但如果父进程代码写得烂（有 Bug），一直不去回收，这个PID就会一直卡在 Z 状态。

T状态-暂停

Load Average平均负载

如果load average为2.0，对于单核CPU：说明1 辆车在过收费站，还有 1 辆在排队，系统已经开始卡顿。对于四核CPU，如果 Load 是 2.0，说明 4 条车道只有 2 条在使用，另外 2 条空闲，系统非常流畅

另外，平均负载跟CPU利用率不是一回事，Load Average 衡量的是‘系统整体资源的紧张程度’，而不是单纯的 CPU 繁忙程度

CPU 使用率只有 5%，但 Load 飙到了 100。 > 这绝对是因为系统里出现了海量的 D 状态进程。比如底层的云硬盘故障、NFS 挂载点断开、或者是疯狂的顺序写打满了磁盘的 IOPS。进程全卡在等磁盘数据返回，导致 CPU 闲得发慌，但系统的 Load 却直接爆炸

面试官问：那如果你遇到了高负载，你的排查思路是什么？

如果是CPU 的 `us`（用户态）或 `sy`（内核态）极高，那就是典型的CPU密集型，这时候需要查看是哪个进程吃光了CPU，如果是`iowait`极高，就说明是典型的I/O密集型，说明磁盘利用率非常高

上下文切换

面试官问：进程切换和线程切换有什么区别？开销一样大吗？

进程切换不仅要跨越用户态和内核态，需要保存和恢复寄存器，而且它必须切换虚拟内存空间（页表）。一旦页表切换，CPU 内部缓存的虚拟地址到物理地址的映射就会失效。新进程开始运行后，需要重新遍历页表填充 TLB，这个过程非常耗时

这是现代微服务的主流模型。因为同一个进程内的多个线程共享同一个虚拟内存空间。所以线程切换时，虚拟内存和 TLB 缓存完全不需要动，只需要切换线程私有的寄存器和栈即可。开销大大降低

什么是中断上下文切换？

中断上下文切换是CPU响应外部事件的强制性机制，这是优先级最高的切换。当网卡收到海量数据包触发硬件中断时，CPU 哪怕正在执行极其关键的业务代码，也必须立刻打断，切换到内核的中断处理程序去收包

模块二：资源模型

内存管理

面试官问：服务器 `free` 内存所剩无几，但系统其实并不卡，业务也没报错，这是为什么？Cache 和 Buffer 有什么区别？

在 Linux 的世界里，空闲的内存就是被浪费的内存

当我们敲下 `free -m` 时，真正能代表系统可用内存的，绝对不是 `free` 字段，而是 `available` 字段

cache(页缓存)：加速文件读取，业务频繁读取本地的日志文件，内核会把文件内容缓存在内存里，下次读直接走内存，速度快上万倍

buffer(缓冲区)：通常用于暂存输入/输出数据，例如从磁盘读数据时，先读入缓冲区，再传给应用程序；写数据时，先写入缓冲区，再批量写入磁盘

swap分区：操作系统将磁盘空间模拟成内存的一种机制，当物理内存（RAM）不足时，内核会把暂时不用的内存页换出到磁盘的 swap 空间，从而释放物理内存给当前需要的进程；当需要访问那些被换出的页时，再将其换入回内存。防止因内存耗尽而导致系统通过 OOM Killer 强制杀死进程，提供一定的过载保护

面试官问：为什么大型云原生架构（Kubernetes）和高并发数据库（Elasticsearch）强制关闭 Swap？

1. 对于高并发应用，每次换入/换出都可能增加几毫秒甚至几十毫秒的延迟，导致请求超时或吞吐量急剧下降
2. kubernetes 通过 cgroups 对容器内存进行硬限制（`memory.limit_in_bytes`）。如果节点开启了 Swap，一个容器可能通过换出部分内存来“假装”自己未超限，但实际上物理内存已被占用，且换出的页仍在磁盘上。这导致：容器实际性能严重受损，但监控可能显示内存使用未超限，造成误判，节点可能报告大量可用内存（实际部分已换出），导致调度更多 Pod，最终引发整体性能雪崩
3. 数据库通常有自己的内存管理策略，希望尽可能将热点数据保留在内存中。Swap 的介入会打乱这种控制，使数据库无法有效利用内存
4. 现代云原生架构中，物理内存已经足够大，而且可以弹性伸缩

CPU架构

用户态：普通应用程序运行在此模式，权限受限，不能直接访问硬件或内核数据结构，只能通过系统调用请求内核代为操作

内核态：操作系统内核运行在此模式，拥有最高权限，可以执行任何指令，访问任何内存地址

为什么说系统调用很慢？

因为会有用户态到内核态的模式切换，CPU 必须保存当前用户态的寄存器上下文

硬中断是硬件通知CPU的方式，要求极短处理时间，运行在关中断环境下。

软中断是内核触发的下半部处理机制，用于处理耗时任务，运行在开中断环境下，可被硬中断抢占

磁盘I/O

IOPS（每秒输入输出次数）：衡量的是磁盘一秒钟能响应多少次独立的读写请求

吞吐量/带宽（Throughput）：衡量的是一秒钟能传输多少 MB 的数据

模块三：网络I/O

面试官问：如果有一个 Web 服务器，同时有 1 万个用户连进来。如果我们用传统的阻塞 I/O（BIO，来一个连接开一个线程），系统会崩溃吗？为什么？

在 Linux 中，每个线程默认要占用大约 2MB 的甚至更多的栈内存，1 万个线程直接吃掉几十 GB 内存，这极易引发 OOM，而且还会有大量的上下文切换，为了解决高并发，我们必须把 socket 设置为非阻塞 I/O（NIO），配合单线程或少量线程来玩。只有一个服务员，他不在一桌死等，而是不停地在大厅里跑，问每一桌‘你点好了吗？’，没点好就立刻问下一桌。但这又引发了新问题：CPU 疯狂空转轮询，依然极其浪费资源

I/O多路复用

select的核心思想：让内核代为监视多个文件描述符，当其中任何一个描述符准备好进行 I/O 时（可读、可写），内核通知用户进程

epoll，先来说说红黑树，因为在高并发场景下，连接的建立和断开极其频繁。红黑树作为一种自平衡二叉查找树，它的增、删、改、查时间复杂度都是极其稳定的 $O(\log N)$

epoll 通过在内核中维护一个事件表（使用红黑树管理所有监视的描述符），而不是像 **select** 那样每次传入一个集合
当描述符就绪时，内核通过回调机制将该描述符添加到就绪链表中，无需遍历整个监视集合
用户调用 **epoll_wait** 时，只需从就绪链表中取出已就绪的描述符，复杂度 $O(1)$

介绍Nginx

Nginx 采用了极其优雅的 ‘一个 Master + 多个 Worker’ 的多进程架构

Master 进程: 它不负责处理具体的网络请求。它只负责读取配置文件、管理 **worker** 进程，如果某个 **worker** 崩溃了，**Master** 会瞬间拉起一个新的
worker 进程: 通常我们会把 **worker** 进程的数量设置得和 CPU 核心数完全一致，并且利用 **worker_cpu_affinity** 把每个 **worker** 进程死死绑定在某个特定的 CPU 核心上，每个 **worker** 进程内部都是单线程的，但它里面跑着一个基于 **epoll**（ET 边缘触发模式）+ 非阻塞 **I/O** 的无限循环。它绝不死等任何一个连接，只要有数据就处理，没数据就切走

面试官：讲讲 Nginx 作为反向代理，有哪些常用的负载均衡算法？什么场景下该用哪种？”

轮询
权重
IP Hash: 如果后端的业务应用有 **Session**（会话）状态，比如用户登录后 **Session** 存在了服务器 **A** 上。如果下一个请求被分配到了服务器 **B**，用户就会被迫重新登录。使用 **IP Hash**，可以保证同一个用户的 **IP** 永远被路由到同一台后端服务器上，解决 **Session** 共享痛点
最少连接: 智能把请求转发给当前活跃连接数最少的后端服务器，适合长连接业务

云原生内容

三大核心

不可变基础设施: 如果这个容器出了故障，或者需要更新代码，云原生的做法是：直接把它杀掉（销毁），然后用最新的镜像重新拉起一个新的容器

声明式API: 这就是云原生和 K8s 的灵魂。不再写步骤，只写一个 YAML 配置文件，告诉系统“我想要什么最终状态 (What) ”

微服务架构与松耦合: 云原生提倡将巨大的单体拆分成极小的**微服务**，支付是一个独立的容器，弹幕是一个独立的容器，它们各自独立部署，互不影响

K8s架构-控制平面

kube-apiserver: 它是整个 **K8s** 集群中唯一对外暴露接口的组件。无论是你敲下 **kubectl** 命令，还是集群内部其他组件互相通信，都必须先经过它的身份认证和权限审查
etcd: 一个分布式的、极其可靠的键值对（**Key-Value**）数据库
kube-controller-manager: 管理器，对比“期望状态”和“实际状态”。如果发现期望是 5 个 **Nginx**，但实际只有 4 个存活，它会立刻生成一个“创建 1 个新 **Nginx**”的任务，实现自愈功能
kube-scheduler: 当 **apiserver** 收到创建一个新应用的请求时，**scheduler** 会看一眼全厂的资源（哪台机器 CPU 空闲？哪台机器内存够大？），经过一套极其复杂的打分算法，最终决定把这个任务派发给哪台具体的 **worker** 节点

K8s架构-工作节点

kubelet: 听 **apiserver** 的命令。一旦 **scheduler** 把某个任务分配到了这台机器上, **kubelet** 就会立刻接旨, 跑去仓库拉取 **Docker** 镜像, 然后把容器启动起来。它还负责定时向大脑汇报本节点的心跳和资源使用情况

kube-proxy: 负责处理这个节点上所有进出的网络流量, 它是 **K8s** 服务发现和四层负载均衡的执行者, 负责把打到集群某个 **IP** 上的流量, 精准地转发到具体的容器里

Container Runtime: 比如以前常用的 **Docker**, 或者现在 **K8s** 默认的 **containerd**。它只负责最底层的苦力活: 根据 **kubelet** 的指令, 把容器的进程 (**namespaces** 和 **cgroups**) 隔离并运行起来

网络

面试官问: 在 **K8s** 里, **Pod** 之间为什么不能直接用 **IP** 地址互相通信? 为什么非要引入一个 **Service** 概念?

如果一个 **Node** 节点宕机了, **K8s** 会在另一个节点上重新拉起这个 **Pod**, 但它的 **IP** 地址会发生彻底的改变! 如果前端服务直接把后端 **Pod** 的 **IP** 写死在代码里, 一旦后端 **Pod** 重启, 前端就彻底瘫痪了

Service 会提供一个永远固定不变的虚拟 **IP** (**ClusterIP**) 和内部 **DNS** 域名。前端服务只需要请求这个固定的名字, 剩下的事它不用管

Service 背后会通过标签选择器 (**Label Selector**) 自动关联那一批真实存活的 **Pod**。当你请求 **Service** 时, 它会在网络层 (**TCP/UDP**, 即四层) 通过轮询或随机算法, 把流量转发给后端的 **Pod**

主要是处理东西流量

Ingress 解决了什么痛点?

管理员写一份 **Ingress** **YAML**, 声明: ‘把访问 **a.com** 的流量转给 **ServiceA**, 把 **b.com** 转给 **ServiceB**’

外部流量只需要打到这一个 **Nginx** 上 (只占用一个公网 **IP** 和 **80/443** 端口), **Nginx** 就会通过七层 **HTTP** 协议的 **Host** 头部和 **Path** 路径, 精准地把流量反向代理到后端的各个 **Service** 上。

主要是处理南北流量

Docker 容器和传统的虚拟机到底有什么区别? 为什么现在大家都爱用容器?

虚拟机跑一个 **web** 服务, 你需要给它虚拟出一整套完整的硬件 (**CPU**、内存), 并且还要在里面安装一个极其庞大的完整操作系统

容器可以说是更轻量级的虚拟机, 秒级启动, 它只是利用了 **Linux** 内核底层的 **Namespace** (做环境隔离) 和 **cgroups** (做资源限制) 技术, 把进程圈了起来。容器将代码、运行时、系统工具、系统库都打包在一起。开发环境、测试环境、生产环境运行的镜像是一致的, 彻底消除了环境差异导致的故障。

你在简历里写了用 **Docker Compose** 编排监控组件, 那 **Docker Compose** 和 **K8s** 有什么区别? 什么时候该用哪个?

Docker Compose 是一个极其轻量级的单机编排工具。我在做这个项目时, 为了让 **Prometheus**、**Grafana** 等组件能在一台服务器上一键启停, 且解决网络互通问题, 写一个 **docker-compose.yml** 是最高效的

现在是周五晚上 8 点, 抖音的一个核心活动上线。突然, 你的 **Prometheus** 监控大盘一片爆红, **Alertmanager** 给你发了最高级别的 **P0** 告警。前端业务团队反馈, 大量用户在请求我们用 **Python** 写的核心后端接口时, 页面直接报错 **504 Gateway Timeout**。

你立刻通过命令行排查，发现 Kubernetes 集群里的 Python API Pod 状态全都是 `Running`（并没有崩溃或重启）。但是你把视线转移到底层的物理机时，敲下 `top` 命令，看到了极其诡异的一幕：

这台机器是 16 核 CPU，CPU 的 `us`（用户态使用率）只有不到 10%，整体感觉 CPU 闲得发慌。但是，系统 1 分钟的 Load Average 飙到了 85，CPU 的 `wa`（iowait 等待）指标极高

面试官问：既然 Pod 都在正常 `Running`，用户看到的 `504 Gateway Timeout` 到底是谁返回的？流量从用户手机，一路上经过了 K8s 的哪些网络组件才到达你的 Python Pod？

Pod 处于 `Running` 状态，只是说明容器没死、进程还在。但这不代表进程有能力处理请求。当用户的 HTTP 请求顺着 `Ingress -> Service` 打到 `Python Pod` 时，`Python` 迟迟不返回结果。`Ingress` 等待的时间超过了它配置的超时上限（比如 60 秒），于是 `Ingress` 掐断了连接，绝望地向用户返回了 `504 Gateway Timeout`

面试官问：结合你在 `top` 命令里看到的“低 CPU 使用率、极高的 Load Average、高 `wa`”这三个指标，请用底层原理给我翻译一下，这台物理机到底遭遇了什么瓶颈？里面的 Python 进程现在正处于什么状态？

cpu 利用率低，load average 高，iowait 高，说明 cpu 的压力并不大，它只是处于空转等待状态，因为 load 的计算除了 cpu，还计算了阻塞进程的情况，此时说明磁盘的 I/O 很高，有进程正在一直读磁盘，python 进程正处于阻塞状态

GitOps + ARgo CD 架构

在 `GitOps` 的世界里，`SRE` 绝对不允许直接去敲命令修改生产环境。你想加一台服务器？你想修改 `Nginx` 配置？统统去改 `Git` 仓库里的 `YAML` 文件，然后提交一个 `Pull Request (PR)`。一旦代码合并，系统会自动把 `Git` 里的状态同步到线上

`Git` 仓库里写着“`Nginx` 版本 `v2`”，但集群里当前跑的是 `v1`。监工说：“不行，`Git` 是老规矩！”于是它主动在集群内部把版本升级到 `v2`

某个 `SRE` 偷偷登录集群，把原本 5 个容器删成了 2 个。监工一看：“不对啊！`Git` 仓库里写的明明是 5 个！”它会立刻触发警报，并强行把容器数量拉回到 5 个